

Tóm tắt Spring Data JPA cho người mới học

Spring Data JPA là một phần của Spring Data, giúp đơn giản hóa việc tương tác với cơ sở dữ liệu bằng cách giảm thiểu mã boilerplate cho các thao tác CRUD (Create, Read, Update, Delete) và cung cấp các tính năng mạnh mẽ khác.

1. ORM và JPA Cơ bản

- **ORM (Object-Relational Mapping):** Kỹ thuật ánh xạ đối tượng Java với bảng cơ sở dữ liệu quan hệ, giúp thao tác dữ liệu dưới dạng đối tượng thay vì SQL thuần. Giải quyết sự khác biệt giữa mô hình đối tượng và mô hình quan hệ.
- **JPA (Java Persistence API):** Là một **đặc tả chuẩn** của Java cho ORM. Nó định nghĩa các API và chú thích (`@Entity` , `@Id` , `@GeneratedValue`) để quản lý các Entity và vòng đời của chúng (New, Managed, Detached, Removed). `EntityManager` là giao diện chính để tương tác với Entity và `Persistence Context` quản lý các Entity đang hoạt động.

2. Spring Data JPA

- **Mục tiêu:** Giảm mã boilerplate, tăng năng suất, dễ kiểm thử và hỗ trợ các tính năng nâng cao.
- **@Repository** : Chú thích đánh dấu một giao diện là Repository, cho phép Spring tự động phát hiện và quản lý. Đồng thời, nó giúp chuyển đổi các ngoại lệ cơ sở dữ liệu thành `DataAccessException` của Spring.
- **Repository Interfaces:**
 - `CrudRepository` : Cung cấp các phương thức CRUD cơ bản.
 - `PagingAndSortingRepository` : Mở rộng `CrudRepository` , thêm khả năng phân trang và sắp xếp.
 - `JpaRepository` : Giao diện phổ biến nhất, kế thừa từ `PagingAndSortingRepository` , cung cấp đầy đủ các chức năng CRUD, phân trang, sắp xếp và các phương thức JPA cụ thể.
- **Query Methods (Derived Query Methods):** Spring Data JPA có thể tự động tạo truy vấn từ tên phương thức của bạn (ví dụ: `findByFirstNameAndLastName`). Điều này rất tiện lợi

cho các truy vấn đơn giản.

- **@Query Annotation:** Cho phép bạn viết các truy vấn tùy chỉnh bằng JPQL (Java Persistence Query Language) hoặc Native SQL khi các Query Methods không đủ linh hoạt. Sử dụng `@Param` để truyền tham số.
- **Pagination & Sorting:** Dễ dàng thực hiện phân trang và sắp xếp bằng cách truyền đối tượng `Pageable` và `Sort` vào các phương thức Repository. Kết quả trả về là đối tượng `Page<T>` chứa dữ liệu và thông tin phân trang.
- **Projections:** Cho phép bạn chọn một tập hợp con các thuộc tính của Entity hoặc ánh xạ kết quả truy vấn sang DTO (Data Transfer Object) để tối ưu hiệu suất và tách biệt mô hình.
- **Specifications:** Công cụ mạnh mẽ để xây dựng các truy vấn động, tái sử dụng và an toàn kiểu dựa trên JPA Criteria API. Hữu ích cho các tìm kiếm phức tạp với nhiều điều kiện.
- **Query by Example (QBE):** Cách đơn giản để xây dựng truy vấn động dựa trên một đối tượng Entity mẫu. Phù hợp cho các tìm kiếm đơn giản với điều kiện `EQUAL`.

3. Kết nối thực tế trong dự án

- **@Transactional :** Đảm bảo tính toàn vẹn dữ liệu bằng cách quản lý giao dịch. Các thao tác trong một phương thức `@Transactional` sẽ được commit hoặc rollback cùng nhau. Nên đặt ở tầng Service Layer.
- **Auditing:** Tự động theo dõi và ghi lại thông tin người tạo/sửa đổi và thời gian tạo/sửa đổi Entity thông qua các chú thích (`@CreatedBy` , `@CreatedDate` , `@LastModifiedBy` , `@LastModifiedDate`) và `AuditorAware` bean.
- **Exception Handling:** Xử lý ngoại lệ một cách nhất quán. Spring chuyển đổi các ngoại lệ cơ sở dữ liệu thành `DataAccessException` . Nên sử dụng `@ControllerAdvice` và `@ExceptionHandler` để xử lý ngoại lệ toàn cục trong ứng dụng RESTful.
- **Service Layer và Cấu trúc dự án:** Service Layer chứa logic nghiệp vụ chính, điều phối các thao tác Repository và quản lý giao dịch. Cấu trúc dự án điển hình bao gồm các package `controller` , `service` , `repository` , `model` , `config` , `exception` để tách biệt các mối quan tâm.

- **Locking (Optimistic & Pessimistic):**

- **Optimistic Locking:** Sử dụng cơ chế versioning (`@Version`) để phát hiện xung đột đồng thời. Hiệu suất cao, phù hợp cho môi trường ít xung đột. Ném `OptimisticLockingFailureException` khi có xung đột.
- **Pessimistic Locking:** Khóa bản ghi trong cơ sở dữ liệu ngay lập tức (`LockModeType.PESSIMISTIC_WRITE` , `PESSIMISTIC_READ`) để ngăn chặn các transaction khác truy cập. Đảm bảo tính toàn vẹn dữ liệu cao nhất, phù hợp cho môi trường nhiều xung đột nhưng có thể ảnh hưởng đến hiệu suất.